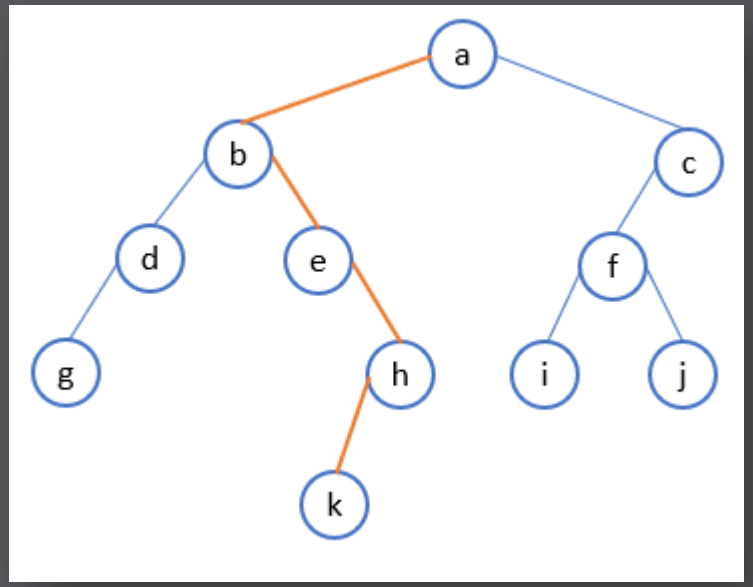
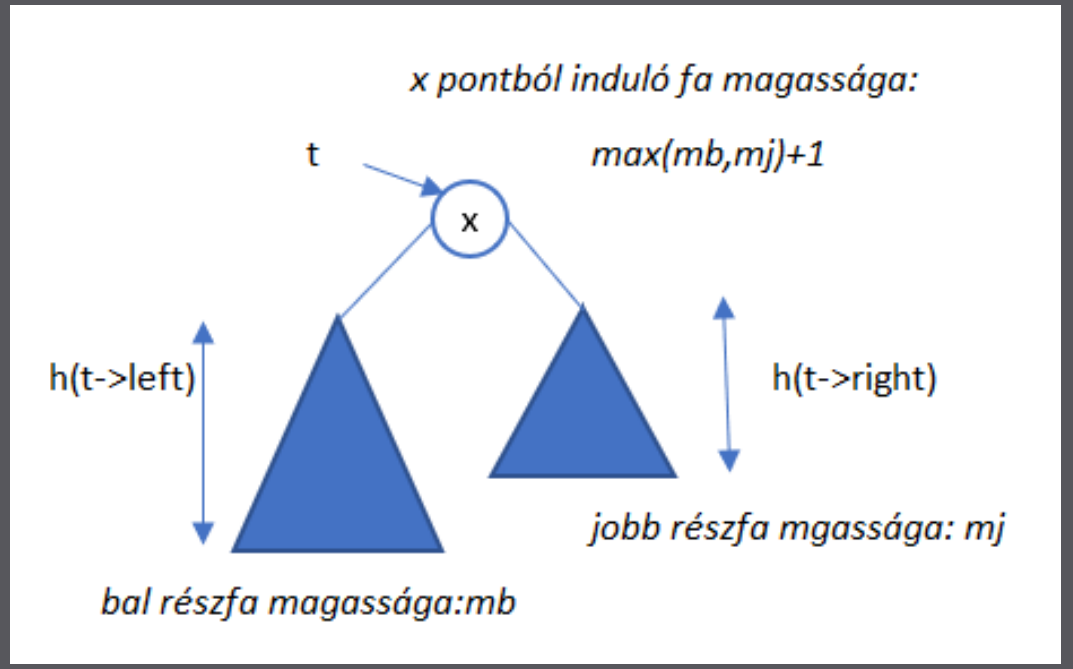
Bináris fa magassága

- Mit értünk magasság alatt:
  a fa egyik leghosszabb ágának a hossza
  (az ág éleinek száma)
- Üres fa magasságát -1-nek értelmezzük.
- Egy pontból álló fa magassága: 0
- Két pontból álló fa magassága: 1
- Példában szereplő fa magassága: 4



Hogyan oldhatjuk meg?

- Ötlet: tetszőleges x pont magassága a következőképpen oldható meg rekurzívan:
  - Határozzuk meg x bal részfájának magasságát, legyen ez mb.
  - Határozzuk meg jobb részfájának a magasságát, legyen ez mj.
  - Ebből x magassága:
    max(mb,mj) + 1
  - Üres fa magassága: -1



Megoldás

h(t:Bintree):Z

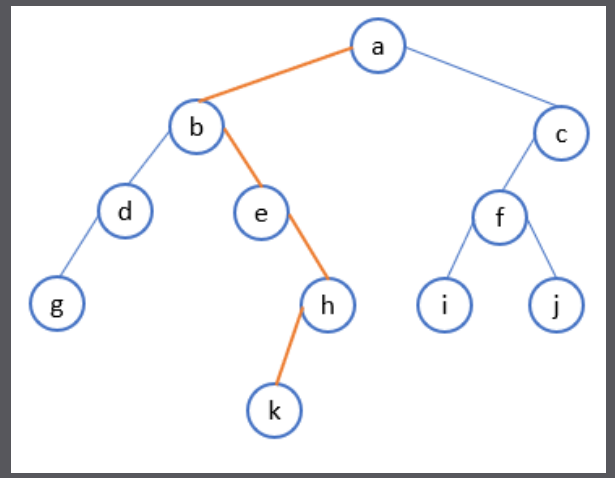
t ≠ 0	
mb:= h(t->left)	return -1
mj:= h(t->right)	
return max(mb,mj)+1	

Esetek ellenőrzése:

üres fa (t=0)

egy pontból álló fa: t → a

két pontból álló fa: t → a, b



Levelek számának meghatározása rekurzív algoritmussal

- Amikor az előbbi ötlet nem válik be:
  - Ha ismerjük egy adott pont bal részfájában a levelek számát: cb
  - Ismerjük a jobb részfájában a levelek számát: cj
  - Akkor az adott pontból induló fa leveleinek száma:
    cb + cj
  - Az üres fa leveleinek száma: 0

levSzam(t:Bintree):N

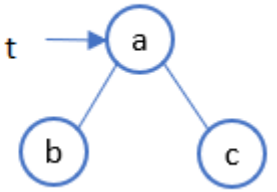
t ≠ 0	
cb:= levSzam(t->left)	return 0
cj:= levSzam(t->right)	
return cb+cj	

Esetek ellenőrzése:

üres fa (t=0)

egy pontból álló fa: t → a

két levelű fa:



Nézzük így:

levSzam(t:Bintree):N

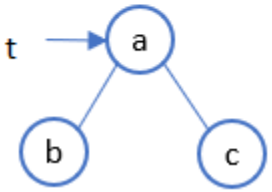
t ≠ 0	
cb:= levSzam(t->left)	return 0
cj:= levSzam(t->right)	
return cb+cj+1	

Esetek ellenőrzése:

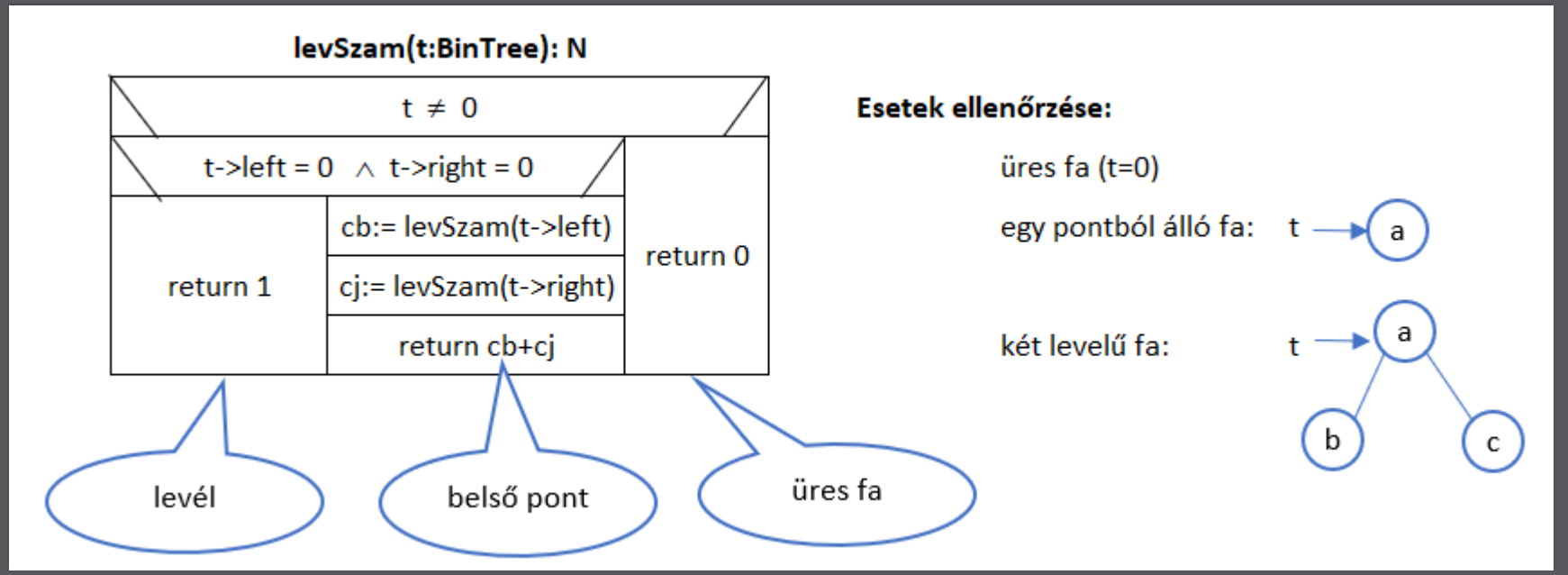
üres fa (t=0)

egy pontból álló fa: t → a

két levelű fa:



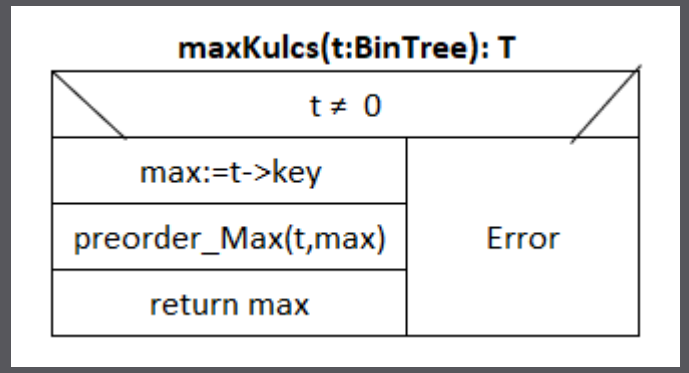
A helyes algoritmus

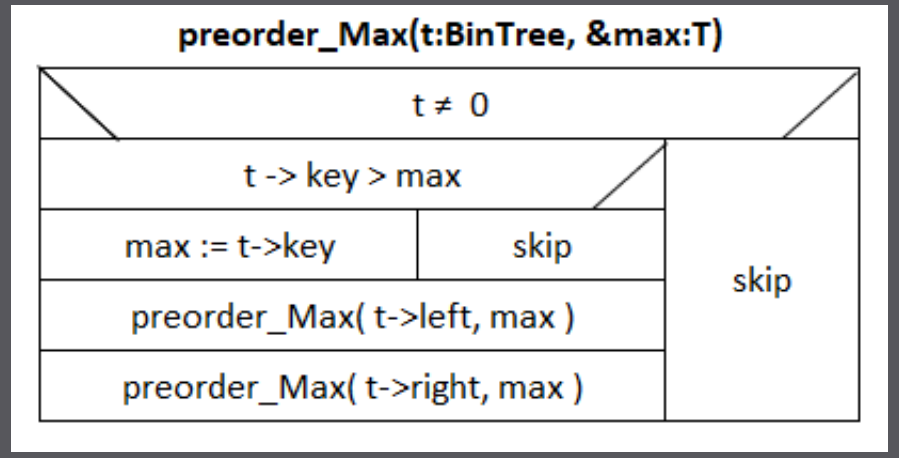


Tanulság: eltérő lehet a fa pontjainak feldolgozása:
levél, egy-gyerekes, két gyerekes.
Üres fa (ha előfordulhat) mindig külön eset.

Bináris fa maximális kulcsának meghatározása

- **Első megoldás:** preorder bejárást követve, egy max változót „viszünk magunkkal”, amelyet folyamatosan frissítünk, az aktuális elem kulcsának megfelően. FONTOS, hogy max referencia szerint átadott paraméter!
- Mivel üres fára nem tudna megoldást adni, az üres fa esetét ki kell szűrni, hibát jelzünk ilyenkor. Kívülről a maxKulcs eljárás hívható, preorder_Max csak egy segéd eljárás. maxKulcs a fa gyökerében lévő kulccsal indítja el a bejárást.





Bináris fa maximális kulcsának meghatározása

- **Második megoldás:** nem paraméterben, hanem visszatérési értékként állítjuk elő a maximumot.
- Ilyenkor is szükség van egy „indító” eljárásra, mely az üres fa esetét kezeli.

maxKulcs(t:BinTree):T

t=0	
Error	return(maxBejár(t))

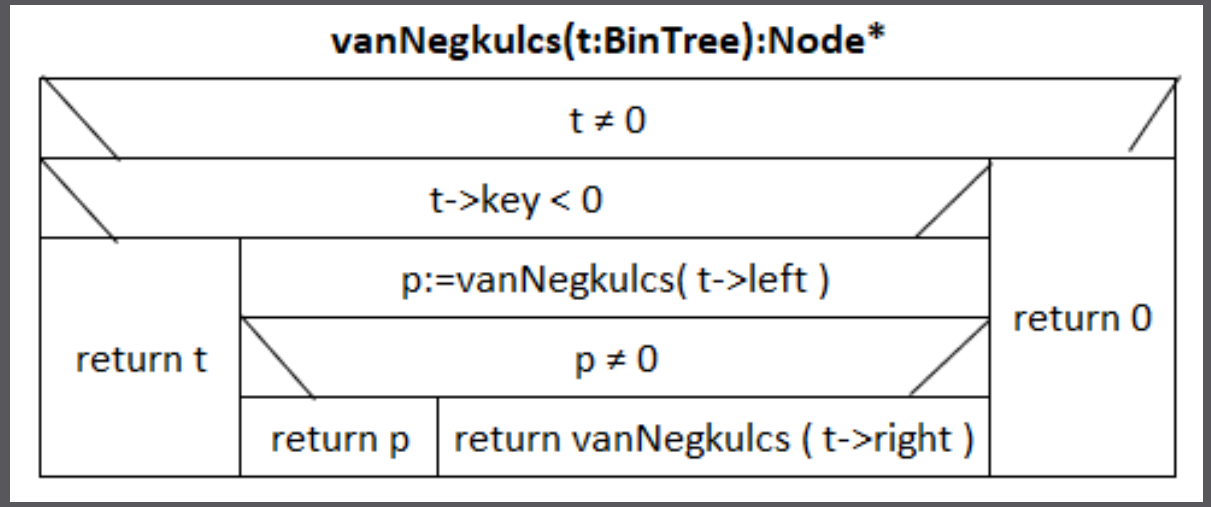
maxBejár(t:BinTree):T

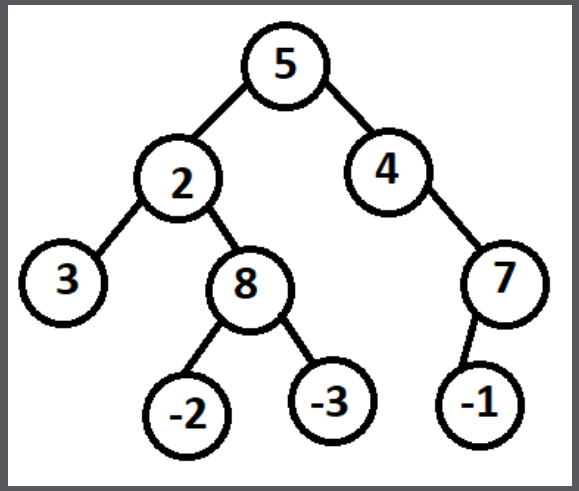
t->left =0 és t->right=0				*levél esete*
	t->left≠0 és t->right≠0	t->left≠0 és t->right=0	t->left=0 és t->right≠0	*belső pontok esetei*
return t->key	max1:=maxBejár(t->left) max2:=maxBejár(t->right) max1:=max(max1,max2)	max1:=maxBejár(t->left)	max1:=maxBejár(t->right)	
	return max(max1,t->key)			

Ennek az algoritmusnak az az előfeltétele, hogy t nem üres fa. Ezért a rekurzív hívást csak a nem üres részfákra hajtjuk végre. Ezért kell a belső pontoknál a háromféle típust külön-külön kezelni.

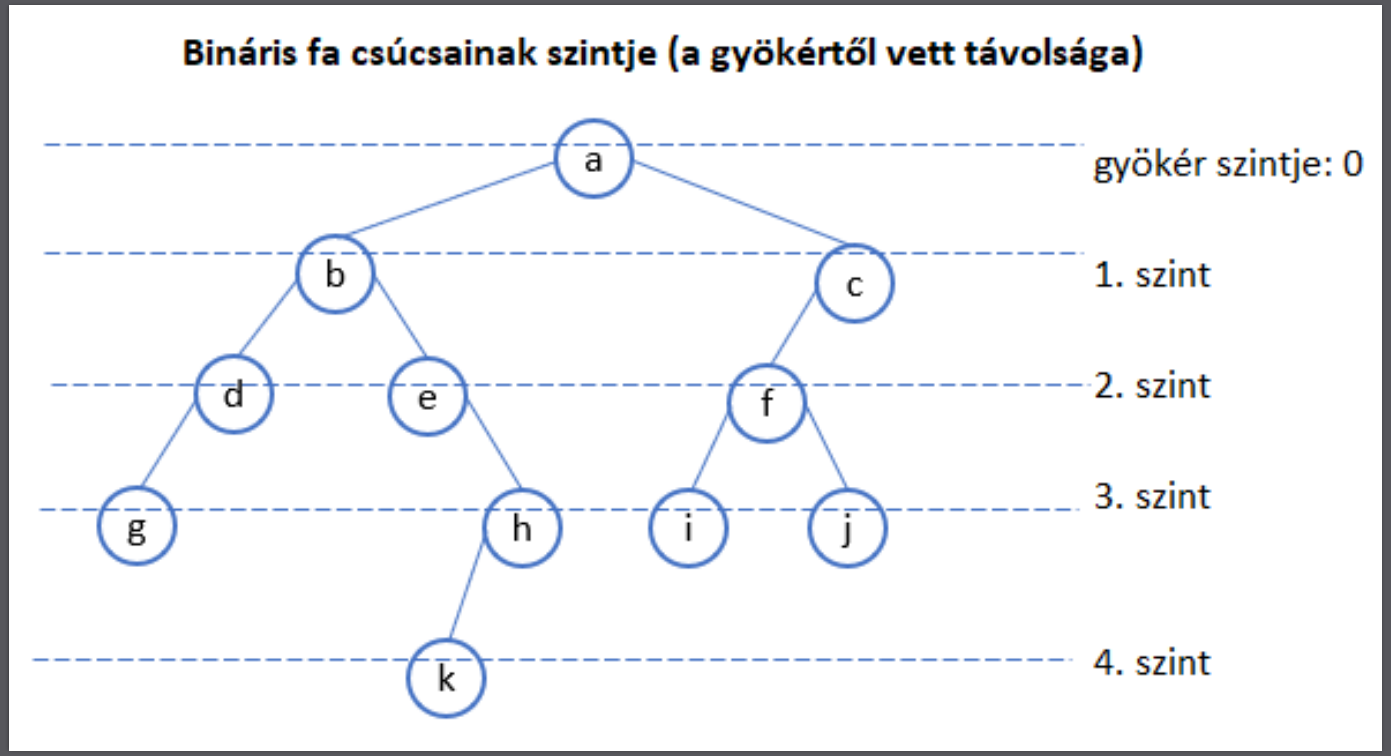
Van-e negatív kulcs egy egész számokat tartalmazó bináris fában, ha igen, adjuk vissza egynek a címét.

- **A feladat keresés.** Függvény alakban készítjük el. Bejárás közben, ha találkozunk egy negatív kulcsú elemmel, leállunk a bejárással, és visszatérünk az elem címével. Ha végigjártuk a fát, és nem találkoztunk negatív kulcsú elemmel, nullát adunk eredményül. Így nincs szükség a keresés logikai eredmény változójára.

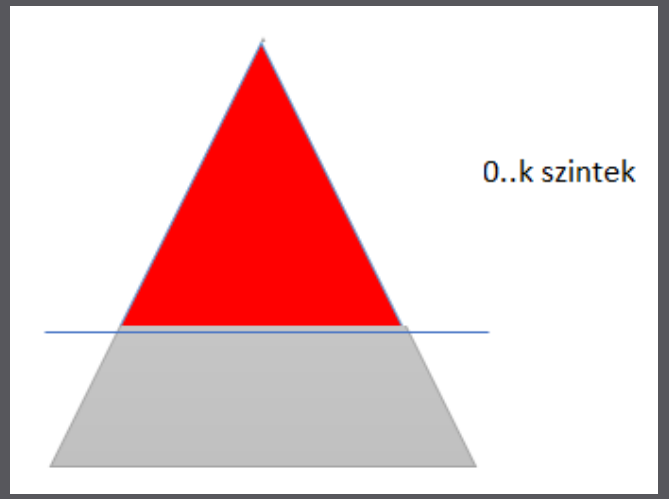




Bináris fa szintjei, szintekkel kapcsolatos feladatok



Hány egy-gyerekes csúcsa van a bináris fának a 0..k szinteken?



egyGyerek(t:Bintree, k: N) : N

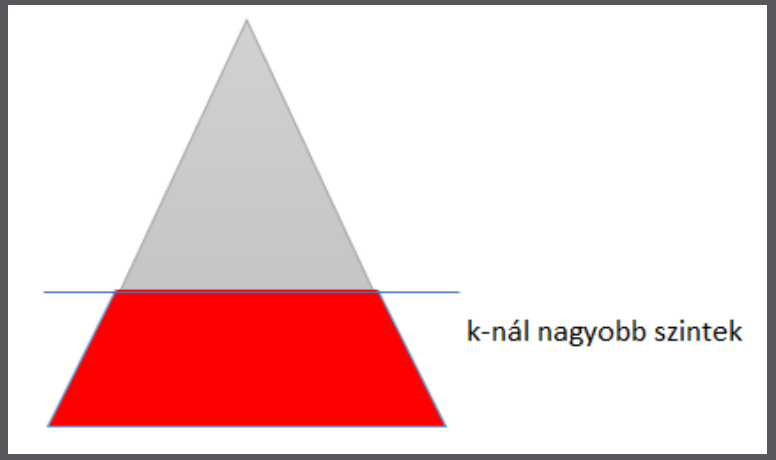
t = 0 vagy k < 0		
return 0	c1:= egyGyerek(t->left,k-1)	
	c2:=egyGyerek(t->right,k-1)	
	(t->left≠0 és t->right=0) vagy (t->left=0 és t->right≠0)	
	return c1+c2+1	return c1+c2

FONTOS:
k érték szerinti paramétere a függvénynek!

Hány egy-gyerekes csúcsa van a bináris fának a k-nál nagyobb szinteken?

egyGyerek(t:Bintree, k: N) : N

t = 0		
return 0	c1:= egyGyerek(t->left,k-1)	
	c2:=egyGyerek(t->right,k-1)	
	k < 0 és ((t->left≠0 és t->right=0) vagy (t->left=0 és t->right≠0))	
	return c1+c2+1	return c1+c2

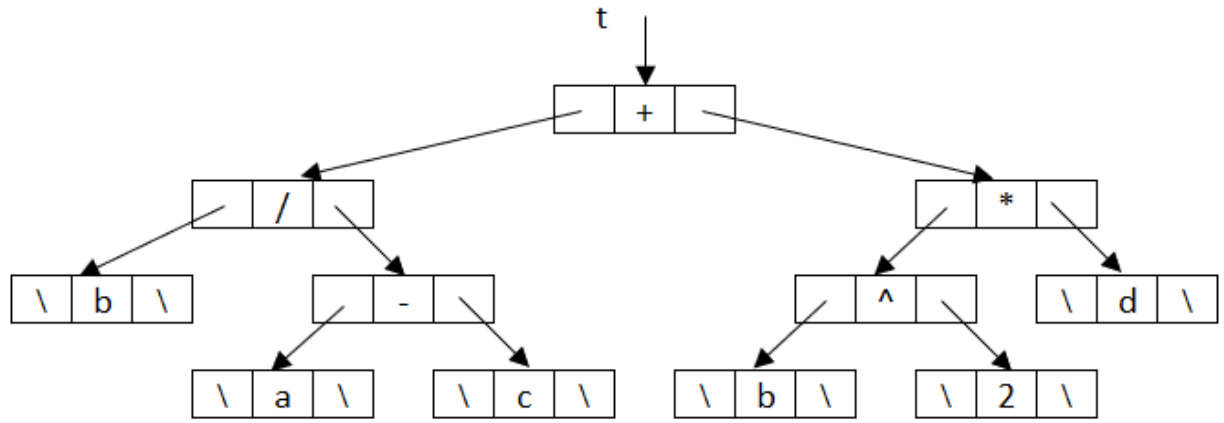


FONTOS:
k érték szerinti paramétere a függvénynek!

Kifejezés fa

- Ha egy aritmetikai kifejezésben két-, vagy egy-operandusú műveleti jelek (operátorok) fordulhatnak elő, a kifejezés ábrázolható egy bináris fával, ezt hívjuk kifejezés fának.
- A kifejezés fa postorder bejárásával előállítható a lengyel forma, és fordítva, a lengyel formából felépíthető a kifejezés fa.

A *b /(a -c)+b ^2 *d* kifejezést ábrázoló kifejezés fa.



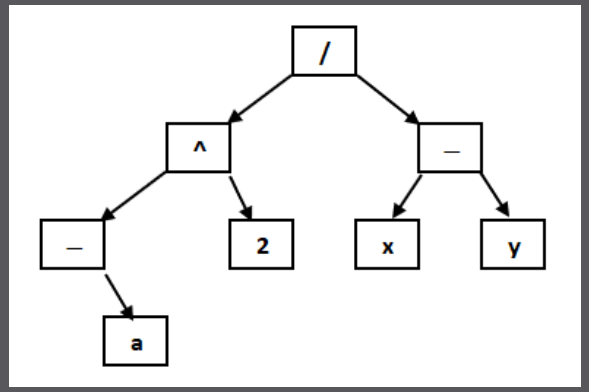
A kifejezés fa postorder bejárása épp a kifejezés lengyel formáját adja:

b a c - / b 2 ^ *d* * +

Milyen sorrendet adna a preoder és az inorder bejárás?

Teljesen zárójelezett alak

- Teljesen (és helyesen) zárójelezett alak: az eredeti helyesen zárójelezett kifejezésnek egy olyan alakja, melyben minden zárójelpár pontosan egy operátort tartalmaz, valamint a hozzá tartozó bal és jobb operandust. Az egymásba ágyazott zárójelpárok határozzák meg a műveletek elvégzésének sorrendjét.



- Példák:

a+b	(a+b)
a+b*c	(a+(b*c))
(a+b)/c	((a+b)/c)
-a^2/(x-y)	(((-a)^2)/(x-y))

- Figyeljük meg, hogy a teljesen zárójelezett kifejezésből könnyen felépíthetjük a kifejezés fát. Meg kell keresni a legkülső zárójelpárban található műveleti jelet, ez lesz a kifejezés fa gyökere. Bal részfáját az opeátor bal oldali operandusából a jobb részfát pedig a jobb operandusból építjük tovább ezen módon. Amikor az operandus már nem zárójelezett kifejezés, akkor belőle egy levél lesz.

Kifejezésfa felépítése a lengyel formából

- Legyen X egy lengyel formájú sorozat, elemei (x_i) vagy az operátorok, vagy az operandusok halmazába tartozik. Készítsük el a kifejezés fáját a lengyel formából.

kifFaÉpít(X) : Node*

V: Stack(); x := read(X)

x ≠ ϵ

t := new Node(x)

Operator(x)	Operandus(x)
t->right := V.pop()	V.push(t)
t->left := V.pop()	
V.push(t)	

x := read(X)

return V.pop()

